

```
#####
#
#   COURS : STATISTIQUES, PROBABILITÉS, MATRICES & ÉQUATIONS DIFF.      #
#   Théorie · Démonstrations · Code Python natif                          #
#                                                                           #
#####
#
# COMMENT LIRE CE COURS ?
#
# Chaque notion suit ce plan :
# [THÉORIE]      – L'idée, pourquoi ça existe
# [DÉFINITION]   – La règle formelle
# [DÉMONSTRATION] – Preuve pas à pas
# [EXEMPLE]      – Exemple concret
# [CODE PYTHON]  – Programme qui illustre la notion
# [PIÈGE]       – Erreur classique à éviter
#
#####
```

PARTIE 1 – STATISTIQUES DESCRIPTIVES

1. MESURES DE TENDANCE CENTRALE

[THÉORIE]

Les statistiques descriptives résument un jeu de données brut en quelques chiffres clés. C'est comme un "résumé exécutif" d'une liste de valeurs. Avant de faire des calculs complexes, on veut savoir : où se situe le "centre" de mes données ? Sont-elles concentrées ou dispersées ?

Trois mesures principales du centre :

- La MOYENNE : le centre de gravité
- La MÉDIANE : la valeur qui coupe la distribution en deux moitiés égales
- Le MODE : la valeur la plus fréquente

[DÉFINITION]

Soit un échantillon $x_1, x_2, \dots, x_n$ de n valeurs.

MOYENNE arithmétique :

$$\bar{x} = (1/n) \cdot \sum_i x_i = (x_1 + x_2 + \dots + x_n) / n$$

MÉDIANE :

- Si n est impair → la valeur centrale après tri croissant
- Si n est pair → moyenne des deux valeurs centrales après tri

MODE :

La valeur (ou les valeurs) qui apparaît le plus souvent dans l'échantillon.

[EXEMPLE]

Données : [3, 7, 7, 2, 9, 4, 7, 1]

Triées : [1, 2, 3, 4, 7, 7, 7, 9]

Moyenne = $(3+7+7+2+9+4+7+1)/8 = 40/8 = 5.0$

Médiane = $(4+7)/2 = 5.5$ ($n=8$, pair → moyenne des 4^e et 5^e)

Mode = 7 (apparaît 3 fois)

[CODE PYTHON]

```
import statistics

donnees = [3, 7, 7, 2, 9, 4, 7, 1]

moyenne = statistics.mean(donnees)
mediane = statistics.median(donnees)
mode = statistics.mode(donnees)

print("Données :", sorted(donnees))
print("Moyenne :", moyenne) # 5.0
print("Médiane :", mediane) # 5.5
print("Mode : ", mode) # 7
```

```
# Sans librairie – calcul manuel de la moyenne
def moyenne_manuelle(lst):
    return sum(lst) / len(lst)

print("\nMoyenne manuelle :", moyenne_manuelle(donnees)) # 5.0
```

[PIÈGE]

La moyenne est très sensible aux valeurs extrêmes (outliers).
 Données : [1, 2, 3, 4, 100] → Moyenne = 22 (trompeur !)
 La médiane = 3 est bien plus représentative ici.
 Règle : si vos données ont des outliers évidents, préférez la médiane.

2. MESURES DE DISPERSION

[THÉORIE]

Deux séries peuvent avoir la même moyenne mais des comportements très différents. [1, 5, 9] et [4, 5, 6] ont toutes les deux la moyenne 5, mais la première est beaucoup plus "étalée". La dispersion mesure cet étalement.

[DÉFINITION]

VARIANCE (σ^2) – mesure l'écart carré moyen à la moyenne :

$$\sigma^2 = (1/n) \cdot \sum_i (x_i - \bar{x})^2$$

ÉCART-TYPE (σ) – racine carrée de la variance, dans la même unité que les données :

$$\sigma = \sqrt{\sigma^2}$$

ÉTENDUE – différence entre le maximum et le minimum :

$$\text{étendue} = \max(x_i) - \min(x_i)$$

ÉCART INTERQUARTILE (IQR) – différence entre le 3^e et 1^{er} quartile :

$$\text{IQR} = Q3 - Q1$$
 (résistant aux outliers, contrairement à l'étendue)

NOTE – Variance corrigée (échantillon) :

$$s^2 = (1/(n-1)) \cdot \sum_i (x_i - \bar{x})^2$$
 ← utiliser quand les données sont un échantillon d'une population plus grande

[DÉMONSTRATION] Pourquoi diviser par (n-1) pour un échantillon ?

Quand on calcule $\bar{x}$ à partir de l'échantillon, on "utilise" une information.
 Il reste n-1 valeurs réellement "libres" (degrés de liberté).
 Diviser par n-1 (correction de Bessel) donne un estimateur non biaisé de σ^2 .
 Si n est grand, la différence entre n et n-1 est négligeable.

[CODE PYTHON]

```
import statistics
import math

donnees = [2, 4, 4, 4, 5, 5, 7, 9]

# Variance population (divise par n)
variance_pop = statistics.pvariance(donnees)

# Variance échantillon (divise par n-1)
variance_ech = statistics.variance(donnees)

# Écart-type
ecart_type_pop = statistics.pstdev(donnees)
ecart_type_ech = statistics.stdev(donnees)

etendue = max(donnees) - min(donnees)

# Quartiles
donnees_tries = sorted(donnees)
n = len(donnees_tries)
Q1 = donnees_tries[n//4]
Q3 = donnees_tries[3*n//4]
IQR = Q3 - Q1

print(f"Données          : {donnees}")
```

```

print(f"Variance (pop) : {variance_pop:.2f}")      # 4.0
print(f"Variance (ech) : {variance_ech:.2f}")      # 4.57
print(f"Écart-type (pop) : {ecart_type_pop:.2f}")  # 2.0
print(f"Écart-type (ech) : {ecart_type_ech:.2f}")  # 2.14
print(f"Étendue : {etendue}")                     # 7
print(f"IQR : {IQR}")                             # 3

# Calcul manuel de la variance population
def variance_manuelle(lst):
    m = sum(lst) / len(lst)
    return sum((x - m)**2 for x in lst) / len(lst)

print(f"\nVariance manuelle : {variance_manuelle(donnees):.2f}") # 4.0

```

[PIÈGE]

Confondre variance population (diviseur n) et variance échantillon (diviseur n-1). En Python : `statistics.variance()` divise par (n-1), `statistics.pvariance()` divise par n.
 Pour l'enseignement scolaire, on utilise souvent n.
 Pour l'inférence statistique, toujours utiliser n-1.

3. VISUALISATION – HISTOGRAMME ET BOÎTE À MOUSTACHES

[THÉORIE]

Un nombre résume, mais une image raconte. L'histogramme montre la distribution des données. La boîte à moustaches (boxplot) résume la distribution en 5 chiffres : min, Q1, médiane, Q3, max.

[DÉFINITION]

RÉSUMÉ EN 5 CHIFFRES :
 Min, Q1 (25ème percentile), Médiane (50ème), Q3 (75ème), Max

OUTLIER (selon règle de Tukey) :
 Valeur < Q1 - 1.5·IQR ou Valeur > Q3 + 1.5·IQR

[CODE PYTHON]

```

# Résumé en 5 chiffres sans librairie externe

def resume_5_chiffres(lst):
    s = sorted(lst)
    n = len(s)

    def percentile(p):
        idx = (n - 1) * p / 100
        lo, hi = int(idx), min(int(idx) + 1, n - 1)
        return s[lo] + (s[hi] - s[lo]) * (idx - lo)

    Q1 = percentile(25)
    med = percentile(50)
    Q3 = percentile(75)
    IQR = Q3 - Q1

    print(f"Min      : {min(s)}")
    print(f"Q1       : {Q1}")
    print(f"Médiane   : {med}")
    print(f"Q3        : {Q3}")
    print(f"Max        : {max(s)}")
    print(f"IQR       : {IQR:.2f}")

    borne_inf = Q1 - 1.5 * IQR
    borne_sup = Q3 + 1.5 * IQR
    outliers = [x for x in s if x < borne_inf or x > borne_sup]
    print(f"Outliers : {outliers}")

donnees = [10, 12, 14, 15, 15, 16, 18, 20, 21, 22, 100]
resume_5_chiffres(donnees)
# 100 sera détecté comme outlier

# Histogramme en mode texte (ASCII)
def histogramme_ascii(lst, bins=5):

```

```

mn, mx = min(lst), max(lst)
largeur = (mx - mn) / bins
print(f"\nHistogramme (intervalles de {largeur:.1f})")
for i in range(bins):
    borne_g = mn + i * largeur
    borne_d = mn + (i + 1) * largeur
    compte = sum(1 for x in lst if borne_g <= x < borne_d)
    barre = "█" * compte
    print(f"[{borne_g:6.1f} – {borne_d:6.1f}] : {barre} ({compte})")

```

```

histogramme_ascii([2,3,3,4,4,4,5,5,5,5,6,6,7,8,9], bins=4)

```

PARTIE 2 – PROBABILITÉS

4. ESPACE DE PROBABILITÉ – LES FONDATIONS

[THÉORIE]

La probabilité répond à la question : "Quelle est la chance que cet événement se produise ?" En programmation, c'est comme demander : "Sur 1 000 000 simulations, combien de fois x se passe-t-il ?" Les axiomes de Kolmogorov formalisent cette idée.

[DÉFINITION]

ESPACE DE PROBABILITÉ $(\Omega, \mathcal{F}, P)$:

Ω = espace fondamental (tous les résultats possibles)
 $\mathcal{F}$ = tribu (ensemble des événements, sous-ensembles de Ω)
 P = mesure de probabilité

AXIOMES DE KOLMOGOROV :

1. $P(A) \geq 0$ pour tout événement A
2. $P(\Omega) = 1$ (certitude)
3. Si $A \cap B = \emptyset$, alors $P(A \cup B) = P(A) + P(B)$ (additivité)

PROPRIÉTÉS DÉRIVÉES :

$P(\emptyset) = 0$
 $P(A^c) = 1 - P(A)$ (événement complémentaire)
 $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (union)

[DÉMONSTRATION] $P(A^c) = 1 - P(A)$

A et A^c sont disjoints ($A \cap A^c = \emptyset$) et $A \cup A^c = \Omega$.

Par l'axiome 3 : $P(A \cup A^c) = P(A) + P(A^c)$

Mais $P(A \cup A^c) = P(\Omega) = 1$ (axiome 2)

Donc $1 = P(A) + P(A^c) \rightarrow P(A^c) = 1 - P(A)$. $\square$

[EXEMPLE]

Lancer un dé à 6 faces équilibré :

$\Omega = \{1, 2, 3, 4, 5, 6\}$

$P(\{k\}) = 1/6$ pour tout k

$A = \text{"obtenir un nombre pair"} = \{2, 4, 6\} \rightarrow P(A) = 3/6 = 0.5$

$B = \text{"obtenir } > 4" = \{5, 6\} \rightarrow P(B) = 2/6 \approx 0.33$

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
 $= 3/6 + 2/6 - P(\{6\}) = 3/6 + 2/6 - 1/6 = 4/6 \approx 0.67$

[CODE PYTHON]

```

import random

# Simulation d'un espace de probabilité – lancer de dé
random.seed(42)

N = 100_000 # nombre de simulations

resultats = [random.randint(1, 6) for _ in range(N)]

# Calcul des probabilités expérimentales
def proba_experimentale(resultats, evenement_func):
    favorables = sum(1 for x in resultats if evenement_func(x))
    return favorables / len(resultats)

```

```

P_pair = proba_experimentale(resultats, lambda x: x % 2 == 0)
P_sup4 = proba_experimentale(resultats, lambda x: x > 4)
P_pair_ou_sup4 = proba_experimentale(resultats, lambda x: x % 2 == 0 or x > 4)

print(f"P(pair) = {P_pair:.4f} (théorie : {3/6:.4f})")
print(f"P(> 4) = {P_sup4:.4f} (théorie : {2/6:.4f})")
print(f"P(pair ou > 4) = {P_pair_ou_sup4:.4f} (théorie : {4/6:.4f})")

# Vérification de  $P(A^c) = 1 - P(A)$ 
P_impair = proba_experimentale(resultats, lambda x: x % 2 != 0)
print(f"\nP(impair) = {P_impair:.4f}")
print(f"1 - P(pair) = {1 - P_pair:.4f}")
print(f"Égale ? : {abs(P_impair - (1 - P_pair)) < 0.01}")

```

[PIÈGE]

"Ces événements sont improbables donc leur union aussi."
 C'est faux dès qu'ils sont nombreux.
 $P(\text{au moins un 6 en 10 lancers}) = 1 - (5/6)^{10} \approx 0.84$ (pas si rare !)

5. PROBABILITÉ CONDITIONNELLE ET THÉORÈME DE BAYES

[THÉORIE]

La probabilité conditionnelle répond à : "Sachant que B s'est produit, quelle est la probabilité que A se produise aussi ?"
 Nouvelle information → mise à jour de la probabilité.
 C'est le fondement du raisonnement bayésien utilisé en IA.

[DÉFINITION]

PROBABILITÉ CONDITIONNELLE (si $P(B) > 0$) :

$$P(A | B) = P(A \cap B) / P(B)$$

INDÉPENDANCE de A et B :

A et B sont indépendants $\iff P(A \cap B) = P(A) \cdot P(B)$
 Équivalent : $P(A | B) = P(A)$ (B n'apporte aucune info sur A)

THÉORÈME DES PROBABILITÉS TOTALES :

Si $B_1, \dots, B_n$ forment une partition de Ω :

$$P(A) = \sum_i P(A | B_i) \cdot P(B_i)$$

THÉORÈME DE BAYES :

$$P(B | A) = P(A | B) \cdot P(B) / P(A)$$

[EXEMPLE – Test médical]

Maladie rare : $P(\text{Malade}) = 0.01$ (1 % de la population)

Test :

$P(\text{Positif} | \text{Malade}) = 0.99$ (sensibilité)
 $P(\text{Positif} | \text{Sain}) = 0.05$ (faux positifs)

Question : si mon test est positif, quelle est la proba d'être malade ?

$$P(\text{Malade} | \text{Positif}) = P(\text{Positif} | \text{Malade}) \cdot P(\text{Malade}) / P(\text{Positif})$$

$$\begin{aligned}
 P(\text{Positif}) &= P(\text{Positif} | \text{Malade}) \cdot P(\text{Malade}) + P(\text{Positif} | \text{Sain}) \cdot P(\text{Sain}) \\
 &= 0.99 \cdot 0.01 + 0.05 \cdot 0.99 \\
 &= 0.0099 + 0.0495 = 0.0594
 \end{aligned}$$

$$P(\text{Malade} | \text{Positif}) = (0.99 \cdot 0.01) / 0.0594 \approx 0.1667$$

→ Seulement 16.7% de chance d'être malade malgré un test positif !

[CODE PYTHON]

Théorème de Bayes – test médical

```

def bayes(P_B, P_A_sachant_B, P_A_sachant_non_B):
    """
    Calcule P(B | A) via le théorème de Bayes.
    B = hypothèse (ex: malade)
    A = observation (ex: test positif)
    """
    P_non_B = 1 - P_B

```

```

P_A = P_A_sachant_B * P_B + P_A_sachant_non_B * P_non_B
P_B_sachant_A = (P_A_sachant_B * P_B) / P_A
return P_B_sachant_A, P_A

# Test médical
P_malade = 0.01
P_pos_si_malade = 0.99
P_pos_si_sain = 0.05

P_malade_si_pos, P_pos = bayes(P_malade, P_pos_si_malade, P_pos_si_sain)

print("=== Test Médical – Bayes ===")
print(f"P(test positif) = {P_pos:.4f}")
print(f"P(malade | test positif) = {P_malade_si_pos:.4f}") # ≈ 0.1667

# Simulation pour vérifier
import random
random.seed(0)

N = 1_000_000
tp = fp = 0
for _ in range(N):
    malade = random.random() < P_malade
    if malade:
        positif = random.random() < P_pos_si_malade
    else:
        positif = random.random() < P_pos_si_sain

    if positif and malade: tp += 1
    if positif and not malade: fp += 1

total_positifs = tp + fp
print(f"\nSimulation ({N} personnes):")
print(f"Vrais positifs : {tp}, Faux positifs : {fp}")
print(f"P(malade | positif) simulée = {tp/total_positifs:.4f}")

```

[PIÈGE]

Le SOPHISME DU PROCUREUR : confondre $P(\text{preuve} \mid \text{coupable})$ et $P(\text{coupable} \mid \text{preuve})$. Si $P(\text{preuve} \mid \text{coupable}) = 0.999$, ça NE VEUT PAS dire $P(\text{coupable} \mid \text{preuve}) = 0.999$. Cela dépend aussi de $P(\text{coupable})$ a priori.

6. VARIABLES ALÉATOIRES ET DISTRIBUTIONS

[THÉORIE]

Une variable aléatoire (VA) est une fonction qui associe un nombre réel à chaque résultat d'une expérience aléatoire. On peut la voir comme une variable dont la valeur n'est connue qu'une fois l'expérience faite.

[DÉFINITION]

VA DISCRÈTE X : prend des valeurs dans un ensemble dénombrable.

Loi de probabilité : $P(X = k)$ pour chaque valeur k

Espérance : $E[X] = \sum_k k \cdot P(X = k)$

Variance : $\text{Var}(X) = E[X^2] - (E[X])^2$

VA CONTINUE X : prend ses valeurs dans un intervalle réel.

Densité de probabilité $f(x)$ telle que $\int f(x)dx = 1$

$P(a \leq X \leq b) = \int_a^b f(x)dx$

Espérance : $E[X] = \int x \cdot f(x)dx$

DISTRIBUTIONS CLASSIQUES :

Uniforme discrète $U(\{1, \dots, n\})$: $P(X=k) = 1/n$

Binomiale $B(n, p)$: $P(X=k) = C(n, k) \cdot p^k \cdot (1-p)^{n-k}$

Poisson $P(\lambda)$: $P(X=k) = e^{-(\lambda)} \cdot \lambda^k / k!$

Normale $N(\mu, \sigma^2)$: $f(x) = (1/\sigma\sqrt{2\pi}) \cdot \exp(-(x-\mu)^2/2\sigma^2)$

Exponentielle $\text{Exp}(\lambda)$: $f(x) = \lambda \cdot e^{-(\lambda x)}$ pour $x \geq 0$

[EXEMPLE – Loi Binomiale]

On lance 10 fois une pièce équilibrée. X = nombre de Pile.

$X \sim B(n=10, p=0.5)$

$E[X] = n \cdot p = 5$ (on attend 5 Pile en moyenne)

$\text{Var}(X) = n \cdot p \cdot (1-p) = 2.5$

$$P(X = 3) = C(10,3) \cdot (0.5)^3 \cdot (0.5)^7 \approx 0.117$$

[CODE PYTHON]

```
import math
import random

# — Loi Binomiale —
def combinaison(n, k):
    return math.factorial(n) // (math.factorial(k) * math.factorial(n - k))

def binomiale_pmf(k, n, p):
    """P(X = k) pour X ~ B(n, p)"""
    return combinaison(n, k) * (p ** k) * ((1 - p) ** (n - k))

n, p = 10, 0.5
print("=== Loi Binomiale B(10, 0.5) ===")
for k in range(n + 1):
    prob = binomiale_pmf(k, n, p)
    barre = "█" * int(prob * 80)
    print(f"P(X={k:2d}) = {prob:.4f} {barre}")

esperance = sum(k * binomiale_pmf(k, n, p) for k in range(n + 1))
variance = sum(k**2 * binomiale_pmf(k, n, p) for k in range(n + 1)) - esperance**2
print(f"\nE[X] = {esperance:.2f} (théorie: {n*p})")
print(f"Var = {variance:.2f} (théorie: {n*p*(1-p)})")

# — Loi Normale – simulation par méthode de Box-Muller —
def normale_sample(mu, sigma):
    """Génère un échantillon N(mu, sigma²) via Box-Muller"""
    u1 = random.random()
    u2 = random.random()
    z = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
    return mu + sigma * z

echantillon = [normale_sample(0, 1) for _ in range(10000)]
moy = sum(echantillon) / len(echantillon)
var = sum((x - moy)**2 for x in echantillon) / len(echantillon)
print(f"\n=== Simulation N(0,1) (n=10 000) ===")
print(f"Moyenne simulée = {moy:.4f} (théorie: 0)")
print(f"Variance simulée = {var:.4f} (théorie: 1)")
```

[PIÈGE]

"La loi normale s'applique toujours." FAUX.
 Les revenus, tailles de fichiers, durées de vie suivent souvent
 des lois log-normales ou exponentielles. Toujours vérifier la distribution
 avant d'appliquer des tests supposant la normalité.

PARTIE 3 – STATISTIQUES INFÉRENTIELLES

7. ESTIMATION ET INTERVALLES DE CONFIANCE

[THÉORIE]

On ne peut pas mesurer toute une population. On mesure un échantillon,
 puis on INFÈRE les propriétés de la population.
 C'est comme déboguer un programme en testant sur des cas représentatifs.
 L'intervalle de confiance donne une plage de valeurs plausibles
 pour le paramètre inconnu de la population.

[DÉFINITION]

ESTIMATEUR de la moyenne : $\bar{x} = (1/n) \cdot \sum x_i$

THÉORÈME CENTRAL LIMITE (TCL) :

Si n est grand ($n \geq 30$ en pratique), quelle que soit la distribution
 de la population, la distribution de $\bar{x}$ est approximativement normale :
 $\bar{x} \sim N(\mu, \sigma^2/n)$

INTERVALLE DE CONFIANCE à $(1-\alpha) \cdot 100$ % pour μ (σ connue) :

$IC = [\bar{x} - z_{\{\alpha/2\}} \cdot \sigma / \sqrt{n}, \bar{x} + z_{\{\alpha/2\}} \cdot \sigma / \sqrt{n}]$

$z_{\{\alpha/2\}}$ = quantile de la loi normale standard

Pour 95% : $z = 1.96$

Pour 99% : $z = 2.576$

Si σ est inconnue (cas pratique) : remplacer σ par s et z par $t_{\{n-1, \alpha/2\}}$
(loi de Student à $n-1$ degrés de liberté)

[EXEMPLE]

On mesure 36 bouteilles. Tailles (cl) : moyenne $\bar{x} = 75.2$, écart-type $s = 2.4$

IC à 95% pour μ :

marge = $1.96 \cdot 2.4 / \sqrt{36} = 1.96 \cdot 0.4 = 0.784$

IC = $[75.2 - 0.784, 75.2 + 0.784] = [74.416, 75.984]$

Interprétation : on est confiant à 95% que $\mu \in [74.4, 76.0]$

[CODE PYTHON]

```
import math
import random

def z_alpha_sur_2(niveau_confiance):
    """Quantile approximatif de la loi normale standard (unilatéral)"""
    # Approximation de Beasley-Springer-Moro pour les valeurs courantes
    niveaux = {0.90: 1.645, 0.95: 1.960, 0.99: 2.576}
    return niveaux.get(niveau_confiance, 1.960)

def intervalle_confiance(echantillon, niveau=0.95):
    """Intervalle de confiance pour la moyenne,  $\sigma$  inconnue (approx normale)"""
    n = len(echantillon)
    xb = sum(echantillon) / n
    s = math.sqrt(sum((x - xb)**2 for x in echantillon) / (n - 1))
    z = z_alpha_sur_2(niveau)
    marge = z * s / math.sqrt(n)
    return xb, xb - marge, xb + marge, marge

# Simulation : population N(75, 2.4²), échantillon de 36
random.seed(7)
echantillon = [random.gauss(75, 2.4) for _ in range(36)]
xb, borne_inf, borne_sup, marge = intervalle_confiance(echantillon, 0.95)

print(f"Moyenne échantillon : {xb:.3f}")
print(f"IC 95% : [{borne_inf:.3f}, {borne_sup:.3f}]")
print(f"Marge d'erreur : ±{marge:.3f}")
print(f"La vraie moyenne (75) est dans l'IC ? : {borne_inf <= 75 <= borne_sup}")

# Vérification du TCL : sur 1000 IC, environ 95% doivent contenir  $\mu=75$ 
vrais_mu = 0
for _ in range(1000):
    ech = [random.gauss(75, 2.4) for _ in range(36)]
    _, bi, bs, _ = intervalle_confiance(ech, 0.95)
    if bi <= 75 <= bs:
        vrais_mu += 1
print(f"\nCouverture empirique : {vrais_mu/10:.1f}% (attendu: 95%)")
```

[PIÈGE]

Un IC à 95% NE SIGNIFIE PAS "il y a 95% de chances que μ soit dans cet IC".
 μ est fixe (inconnu). Ce qui a 95% de probabilité c'est la PROCÉDURE :
si on répétait l'expérience 100 fois, environ 95 des IC construits
contiendraient μ .

8. TESTS D'HYPOTHÈSES

[THÉORIE]

Un test d'hypothèse est un mécanisme de décision :

"Les données observées sont-elles compatibles avec mon hypothèse de départ ?"

C'est comme un débogueur : on suppose "le code est correct" (H_0) et on

cherche des preuves suffisantes pour le réfuter.

[DÉFINITION]

H_0 = hypothèse nulle (ce qu'on cherche à réfuter)
 H_1 = hypothèse alternative (ce qu'on veut prouver)

STATISTIQUE DE TEST : valeur calculée sur l'échantillon
P-VALEUR : $P(\text{observer résultat aussi extrême} \mid H_0 \text{ vraie})$
SEUIL α : niveau de signification (souvent $\alpha = 0.05$)

DÉCISION :
 $p\text{-valeur} < \alpha \rightarrow$ rejeter H_0 (résultat "statistiquement significatif")
 $p\text{-valeur} \geq \alpha \rightarrow$ ne pas rejeter H_0

ERREURS :
Type I (α) : rejeter H_0 alors qu'elle est vraie (faux positif)
Type II (β) : ne pas rejeter H_0 alors qu'elle est fautive (faux négatif)

TEST Z (moyenne, σ connue) :
 $H_0 : \mu = \mu_0$
 $z = (\bar{x} - \mu_0) / (\sigma / \sqrt{n})$
Sous $H_0 : z \sim N(0,1)$

[EXEMPLE]

Fabricant affirme : $\mu = 500\text{g}$. Échantillon de 25 paquets : $\bar{x} = 495\text{g}$, $\sigma = 10\text{g}$.
 $H_0 : \mu = 500$ vs $H_1 : \mu \neq 500$ (test bilatéral, $\alpha = 0.05$)

$z = (495 - 500) / (10/\sqrt{25}) = -5 / 2 = -2.5$
 $|z| = 2.5 > z_{\{0.025\}} = 1.96 \rightarrow$ on rejette H_0
Conclusion : l'affirmation du fabricant est douteuse.

[CODE PYTHON]

```
import math
import random

# — Test Z bilatéral ( $\sigma$  connue) —————

def test_z(echantillon, mu0, sigma):
    """
    Test z bilatéral  $H_0: \mu = \mu_0$  vs  $H_1: \mu \neq \mu_0$ 
    Retourne (z_stat, p_valeur approximée)
    """
    n = len(echantillon)
    xb = sum(echantillon) / n
    z = (xb - mu0) / (sigma / math.sqrt(n))

    # P-valeur approx pour test bilatéral (table normale)
    #  $P(Z > |z|)$  approximé par formule de Abramowitz & Stegun
    def phi_complementaire(t):
        t = abs(t)
        a1 = 0.319381530; a2 = -0.356563782
        a3 = 1.781477937; a4 = -1.821255978
        a5 = 1.330274429; p = 0.2316419
        k = 1 / (1 + p * t)
        poly = (((a5*k + a4)*k + a3)*k + a2)*k + a1*k
        return (1/math.sqrt(2*math.pi)) * math.exp(-0.5*t*t) * poly

    p_valeur = 2 * phi_complementaire(z)
    return z, p_valeur

# Simulation :  $\mu$  réel = 495, mais on teste  $H_0: \mu = 500$ 
random.seed(12)
echantillon = [random.gauss(495, 10) for _ in range(25)]
z_stat, p_val = test_z(echantillon, mu0=500, sigma=10)

print("=== Test Z bilatéral ===")
print(f"x̄ = {sum(echantillon)/25:.2f}")
print(f"Statistique z = {z_stat:.3f}")
print(f"P-valeur = {p_val:.4f}")
print(f"Décision ( $\alpha=0.05$ ) : {'Rejeter  $H_0$ ' if p_val < 0.05 else 'Ne pas rejeter  $H_0$ '})")

# Taux d'erreur de type I : si  $H_0$  est vraie, combien de fois la rejette-t-on ?
rejets = 0
for _ in range(10000):
    ech = [random.gauss(500, 10) for _ in range(25)] #  $H_0$  vraie :  $\mu=500$ 
    _, pv = test_z(ech, mu0=500, sigma=10)
    if pv < 0.05:
```

```

    rejets += 1
print(f"\nTaux erreur type I simulé : {rejets/100:.1f}% (attendu: 5%)")

```

[PIÈGE]

p-valeur < 0.05 ne signifie PAS que l'effet est important, ni que H_1 est vraie à 95%. Cela dit seulement que le résultat est improbable sous H_0 . Avec un très grand n, même un effet minuscule devient "significatif". Toujours accompagner le test d'une mesure de TAILLE D'EFFET.

PARTIE 4 – CORRÉLATION ET RÉGRESSION

9. CORRÉLATION

[THÉORIE]

La corrélation mesure la force et la direction d'une relation LINÉAIRE entre deux variables. C'est une valeur entre -1 et 1. Elle ne mesure PAS la causalité, seulement l'association.

[DÉFINITION]

COVARIANCE entre X et Y :

$$\text{Cov}(X, Y) = (1/n) \cdot \sum_i (x_i - \bar{x})(y_i - \bar{y})$$

COEFFICIENT DE CORRÉLATION DE PEARSON :

$$r = \text{Cov}(X, Y) / (\sigma_X \cdot \sigma_Y) \quad \text{avec } r \in [-1, 1]$$

$r = 1$: corrélation positive parfaite

$r = 0$: aucune corrélation linéaire

$r = -1$: corrélation négative parfaite

CORRÉLATION DE SPEARMAN (rang) :

Remplace les valeurs par leurs rangs, puis calcule r de Pearson.

Mesure les relations monotones (pas forcément linéaires).

Plus robuste aux outliers.

[DÉMONSTRATION] $r \in [-1, 1]$

Par l'inégalité de Cauchy-Schwarz :

$$|\text{Cov}(X, Y)|^2 \leq \text{Var}(X) \cdot \text{Var}(Y) = \sigma_X^2 \cdot \sigma_Y^2$$

Donc $|\text{Cov}(X, Y)| \leq \sigma_X \cdot \sigma_Y$

Donc $|r| = |\text{Cov}(X, Y)| / (\sigma_X \cdot \sigma_Y) \leq 1$. $\square$

[EXEMPLE]

Taille (cm) : [160, 165, 170, 175, 180]

Poids (kg) : [55, 60, 70, 72, 80]

On s'attend à $r > 0$: plus on est grand, plus on tend à peser lourd.

[CODE PYTHON]

```

import math

def pearson(x, y):
    """Coefficient de corrélation de Pearson"""
    n = len(x)
    xb = sum(x) / n
    yb = sum(y) / n
    cov = sum((xi - xb)*(yi - yb) for xi, yi in zip(x, y)) / n
    sx = math.sqrt(sum((xi - xb)**2 for xi in x) / n)
    sy = math.sqrt(sum((yi - yb)**2 for yi in y) / n)
    return cov / (sx * sy) if sx * sy != 0 else 0

def spearman(x, y):
    """Corrélation de Spearman (par rangs)"""
    def rangs(lst):
        triee = sorted(enumerate(lst), key=lambda t: t[1])
        r = [0] * len(lst)
        for rang, (idx, _) in enumerate(triee, 1):
            r[idx] = rang
        return r
    return pearson(rangs(x), rangs(y))

```

```

tailles = [160, 165, 170, 175, 180]
poids   = [55, 60, 70, 72, 80]

r_p = pearson(tailles, poids)
r_s = spearman(tailles, poids)

print(f"Corrélation Pearson : {r_p:.4f}")
print(f"Corrélation Spearman : {r_s:.4f}")

# Interprétation
if abs(r_p) > 0.7:
    force = "forte"
elif abs(r_p) > 0.4:
    force = "modérée"
else:
    force = "faible"
sens = "positive" if r_p > 0 else "négative"
print(f"→ Corrélation {force} et {sens}")

# Nuage de points ASCII
print("\nNuage de points (taille vs poids) :")
print("Poids")
for p_val in range(80, 50, -5):
    ligne = f"{p_val:3d} |"
    for t in tailles:
        # Trouver le poids correspondant
        idx = tailles.index(t)
        if abs(poids[idx] - p_val) <= 2.5:
            ligne += " * "
        else:
            ligne += "   "
    print(ligne)
print("      + " + "-"*25 + " Taille")

```

[PIÈGE]

"Corrélation implique causalité." FAUX.
 Exemple classique : les ventes de glaces et les noyades sont corrélées.
 La cause commune est la chaleur. Toujours chercher une explication causale
 plutôt que de conclure directement depuis la corrélation.

10. RÉGRESSION LINÉAIRE

[THÉORIE]

La corrélation dit "il y a une relation". La régression quantifie cette
 relation sous forme d'une équation : $\hat{y} = a \cdot x + b$.
 Elle permet de PRÉDIRE y à partir de x .

[DÉFINITION]

DROITE DE RÉGRESSION (moindres carrés ordinaires) :

Minimiser $\sum_i (y_i - \hat{y}_i)^2 = \sum_i (y_i - a \cdot x_i - b)^2$

SOLUTIONS (dérivées partielles = 0) :

$a = \text{Cov}(X, Y) / \text{Var}(X) = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$
 $b = \bar{y} - a \cdot \bar{x}$

COEFFICIENT DE DÉTERMINATION R^2 :

$R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$
 $R^2 \in [0, 1]$: part de variance de Y expliquée par X

[DÉMONSTRATION] Formule de a

On minimise $f(a, b) = \sum (y_i - a \cdot x_i - b)^2$.
 Condition nécessaire : $\partial f / \partial b = 0$ et $\partial f / \partial a = 0$

$\partial f / \partial b = -2 \cdot \sum (y_i - a \cdot x_i - b) = 0 \rightarrow b = \bar{y} - a \cdot \bar{x}$

En substituant b dans $\partial f / \partial a = 0$:

$\sum (y_i - a \cdot x_i - (\bar{y} - a \cdot \bar{x})) \cdot x_i = 0$
 $\sum ((y_i - \bar{y}) - a \cdot (x_i - \bar{x})) \cdot x_i = 0$
 $\sum (y_i - \bar{y})(x_i - \bar{x}) = a \cdot \sum (x_i - \bar{x})^2$
 $a = \text{Cov}(X, Y) / \text{Var}(X)$. $\square$

[CODE PYTHON]

```
def regression_lineaire(x, y):
    """Calcule les coefficients a, b de  $\hat{y} = ax + b$  (moindres carrés)"""
    n = len(x)
    xb = sum(x) / n
    yb = sum(y) / n
    cov_xy = sum((xi - xb)*(yi - yb) for xi, yi in zip(x, y)) / n
    var_x = sum((xi - xb)**2 for xi in x) / n
    a = cov_xy / var_x
    b = yb - a * xb
    return a, b

def r_carre(x, y, a, b):
    """Coefficient de détermination R²"""
    yb = sum(y) / len(y)
    ss_res = sum((yi - (a*xi + b))**2 for xi, yi in zip(x, y))
    ss_tot = sum((yi - yb)**2 for yi in y)
    return 1 - ss_res / ss_tot

# Exemple : heures d'étude → note obtenue
heures = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
notes = [40, 45, 50, 55, 65, 60, 70, 75, 80, 85]

a, b = regression_lineaire(heures, notes)
R2 = r_carre(heures, notes, a, b)

print("=== Régression Linéaire ===")
print(f"Droite :  $\hat{y} = \{a:.2f\} \cdot x + \{b:.2f\}$ ")
print(f"R² =  $\{R2:.4f\}$  ( $\{R2*100:.1f\}\%$  de variance expliquée)")

# Prédictions
print("\nPrédictions :")
for h in [3, 7, 12]:
    pred = a * h + b
    print(f" {h} heures d'étude → note prédite :  $\{pred:.1f\}$ ")

# Résidus
print("\nRésidus (réel - prédit) :")
for xi, yi in zip(heures, notes):
    pred = a * xi + b
    residu = yi - pred
    print(f" x= $\{xi:2d\}$  : réel= $\{yi:3d\}$ , prédit= $\{pred:.1f\}$ , résidu= $\{residu:+.1f\}$ ")
```

[PIÈGE]

Extrapoler au-delà de la plage des données observées est risqué.
Une droite parfaite entre $x=0$ et $x=10$ peut être inutile pour prédire $x=100$.
Toujours vérifier que la relation linéaire reste valide sur la plage cible.

PARTIE 5 – SÉRIES TEMPORELLES

11. DÉCOMPOSITION D'UNE SÉRIE TEMPORELLE

[THÉORIE]

Une série temporelle est une suite de valeurs mesurées à intervalles réguliers dans le temps : températures, ventes, cours boursiers...
L'objectif est de comprendre son comportement passé pour mieux prévoir.

[DÉFINITION]

DÉCOMPOSITION ADDITIVE :

$$Y_t = T_t + S_t + R_t$$

T_t = TENDANCE (évolution à long terme)

S_t = SAISONNALITÉ (fluctuations périodiques régulières)

R_t = RÉSIDU / BRUIT (variation aléatoire non expliquée)

DÉCOMPOSITION MULTIPLICATIVE (si les fluctuations croissent avec le niveau) :

$$Y_t = T_t \cdot S_t \cdot R_t$$

MOYENNE MOBILE D'ORDRE k :

$$MM_t = (1/k) \cdot \sum_{j=0}^{k-1} Y_{t-j}$$

Lisse la série et fait ressortir la tendance.

LISSAGE EXPONENTIEL SIMPLE :

$$\hat{Y}_{t+1} = \alpha \cdot Y_t + (1-\alpha) \cdot \hat{Y}_t \quad \text{avec } \alpha \in (0,1)$$

α proche de 1 : on réagit vite aux changements

α proche de 0 : on lisse fortement

[EXEMPLE]

Ventes mensuelles sur 2 ans avec saisonnalité :

Décembre ↑ (fêtes), Janvier ↓ (après-fêtes)

Tendance générale : croissante

Résidu : bruit aléatoire autour de la tendance+saisonnalité

[CODE PYTHON]

```
import math

# — Génération d'une série temporelle synthétique —————
def generer_serie(n=48, seed=0):
    """Tendance + Saisonnalité + Bruit"""
    import random
    random.seed(seed)
    serie = []
    for t in range(n):
        tendance = 100 + 2 * t # +2 par période
        saisonnalite = 20 * math.sin(2*math.pi*t/12) # cycle annuel
        bruit = random.gauss(0, 5)
        serie.append(tendance + saisonnalite + bruit)
    return serie

# — Moyenne Mobile —————
def moyenne_mobile(serie, k):
    """Moyenne mobile centrée d'ordre k"""
    mm = []
    for t in range(len(serie)):
        debut = max(0, t - k//2)
        fin = min(len(serie), t + k//2 + 1)
        mm.append(sum(serie[debut:fin]) / (fin - debut))
    return mm

# — Lissage Exponentiel —————
def lissage_exponentiel(serie, alpha):
    """Lissage exponentiel simple"""
    lisse = [serie[0]]
    for t in range(1, len(serie)):
        lisse.append(alpha * serie[t] + (1 - alpha) * lisse[-1])
    return lisse

# — Application —————
serie = generer_serie(48)
mm12 = moyenne_mobile(serie, 12)
lisse_fort = lissage_exponentiel(serie, alpha=0.1) # très lisse
lisse_rapide = lissage_exponentiel(serie, alpha=0.5) # réactif

print("Série temporelle (48 mois) – premières valeurs :")
print([round(v, 1) for v in serie[:12]])

print("\nMoyenne mobile (ordre 12) – premières valeurs :")
print([round(v, 1) for v in mm12[:12]])

# — RMSE de prédiction (lissage exponentiel) —————
def rmse(reel, pred):
    n = min(len(reel), len(pred))
    return math.sqrt(sum((reel[i]-pred[i])**2 for i in range(n)) / n)

rmse_fort = rmse(serie[1:], lisse_fort[:-1])
rmse_rapide = rmse(serie[1:], lisse_rapide[:-1])
print(f"\nRMSE lissage α=0.1 : {rmse_fort:.2f}")
print(f"RMSE lissage α=0.5 : {rmse_rapide:.2f}")

# — Décomposition manuelle : tendance par MM, puis saisonnalité —
mm = moyenne_mobile(serie, 12)
```

```
residus = [serie[t] - mm[t] for t in range(len(serie))]
print(f"\nVariance résidus après décomposition : {sum(r**2 for r in residus)/len(residus):.2f}")
```

[PIÈGE]

Confondre tendance et saisonnalité mène à de mauvaises prévisions.
Si on prévoit décembre sans tenir compte de la saisonnalité des ventes,
on sous-estimera systématiquement.
Toujours décomposer avant de prévoir.

PARTIE 6 – MATRICES ET ALGÈBRE LINÉAIRE

12. MATRICES – DÉFINITIONS ET OPÉRATIONS

[THÉORIE]

Une matrice est un tableau rectangulaire de nombres. On peut la voir
comme une transformation de l'espace : elle prend un vecteur en entrée
et en produit un autre. C'est le fondement du machine learning,
des systèmes d'équations et des transformations géométriques.

[DÉFINITION]

MATRICE A de taille $m \times n$: m lignes, n colonnes.
 $A = (a_{ij})$ avec $i \in \{1, \dots, m\}$ et $j \in \{1, \dots, n\}$

OPÉRATIONS :

Addition : $(A+B)_{ij} = a_{ij} + b_{ij}$ (mêmes dimensions)
Multiplication : $(AB)_{ij} = \sum_k a_{ik} \cdot b_{kj}$ ($A: m \times p, B: p \times n \rightarrow AB: m \times n$)
Transposition : $(A^T)_{ij} = a_{ji}$

PROPRIÉTÉS :

$AB \neq BA$ en général (non-commutativité)
 $(AB)^T = B^T A^T$
 $(AB)C = A(BC)$ (associativité)

MATRICES SPÉCIALES :

Matrice identité I : $I_{ij} = 1$ si $i=j$, 0 sinon. $AI = IA = A$.
Matrice nulle 0 : toutes entrées = 0
Matrice symétrique : $A = A^T$
Matrice diagonale : $a_{ij} = 0$ pour $i \neq j$

[EXEMPLE]

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$AB = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix}$
 $= \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

$BA = \begin{bmatrix} 5 \cdot 1 + 6 \cdot 3 & 5 \cdot 2 + 6 \cdot 4 \\ 7 \cdot 1 + 8 \cdot 3 & 7 \cdot 2 + 8 \cdot 4 \end{bmatrix}$
 $= \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}$

$AB \neq BA$ ✓ (non-commutativité vérifiée)

[CODE PYTHON]

```
# Algèbre matricielle sans librairie externe
```

```
def afficher_matrice(M, nom="M"):
    print(f"{nom} = ")
    for ligne in M:
        print(" [ " + " ".join(f"{v:6.2f}" for v in ligne) + " ]")
    print()
```

```
def addition(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))]
            for i in range(len(A))]
```

```
def multiplication(A, B):
    m, p, n = len(A), len(A[0]), len(B[0])
    C = [[0.0]*n for _ in range(m)]
    for i in range(m):
```

```

    for j in range(n):
        C[i][j] = sum(A[i][k] * B[k][j] for k in range(p))
    return C

def transposee(A):
    return [[A[i][j] for i in range(len(A))]
            for j in range(len(A[0]))]

def scalaire(s, A):
    return [[s * A[i][j] for j in range(len(A[0]))]
            for i in range(len(A))]

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

afficher_matrice(multiplication(A, B), "AB")    # [[19,22],[43,50]]
afficher_matrice(multiplication(B, A), "BA")    # [[23,34],[31,46]]
afficher_matrice(transposee(A), "AT")

# Vérification : (AB)T = BTAT
ABt = transposee(multiplication(A, B))
BtAt = multiplication(transposee(B), transposee(A))
print("(AB)T == BTAT ?")
egal = all(abs(ABt[i][j] - BtAt[i][j]) < 1e-9
            for i in range(len(ABt)) for j in range(len(ABt[0])))
print("→", egal)    # True

```

[PIÈGE]

On ne peut multiplier A·B que si le nombre de COLONNES de A égale le nombre de LIGNES de B. A(m×p) · B(p×n) = C(m×n).
Inverser l'ordre peut rendre l'opération impossible ou donner un résultat de taille différente.

13. DÉTERMINANT, INVERSE ET SYSTÈMES LINÉAIRES

[THÉORIE]

Le déterminant d'une matrice carrée mesure "l'étirement" opéré par la transformation linéaire associée. Si $\det(A) = 0$, la transformation "écrase" l'espace en une dimension inférieure : A n'est pas inversible. Résoudre $Ax = b$ revient à trouver le vecteur x tel que la transformation A appliquée à x donne b.

[DÉFINITION]

DÉTERMINANT 2×2 :
 $\det\begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$

DÉTERMINANT n×n (développement par cofacteurs, colonne 1) :
 $\det(A) = \sum_i a_{i1} \cdot (-1)^{i+1} \cdot \det(M_{i1})$
 M_{i1} = matrice obtenue en supprimant la ligne i et colonne 1

MATRICE INVERSE (si $\det(A) \neq 0$) :
 $A \cdot A^{-1} = A^{-1} \cdot A = I$
 Pour 2×2 : $A^{-1} = (1/\det(A)) \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$

SYSTÈME LINÉAIRE $Ax = b$:
 Si $\det(A) \neq 0$: unique solution $x = A^{-1}b$
 Si $\det(A) = 0$: zéro ou infinité de solutions

ÉLIMINATION DE GAUSS :
 Algorithme pour résoudre $Ax = b$ en $O(n^3)$
 Transforme [A|b] en matrice échelonnée puis résout par substitution.

[DÉMONSTRATION] Formule inverse 2×2

Soit $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. On cherche X tel que $AX = I$.
 En résolvant les 4 équations, on obtient :
 $X = (1/(ad-bc)) \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
 Vérification directe : $A \cdot X = (1/\det) \cdot \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
 $= (1/\det) \cdot \begin{pmatrix} ad-bc & -ab+ba \\ cd-dc & -cb+da \end{pmatrix}$
 $= (1/\det) \cdot \begin{pmatrix} \det & 0 \\ 0 & \det \end{pmatrix} = I. \quad \square$

[CODE PYTHON]

```

def determinant(M):
    """Déterminant par cofacteurs (récursif)"""
    n = len(M)
    if n == 1:
        return M[0][0]
    if n == 2:
        return M[0][0]*M[1][1] - M[0][1]*M[1][0]
    det = 0
    for j in range(n):
        # Mineur  $M_{ij}$  : supprimer ligne 0 et colonne j
        mineur = [[M[i][k] for k in range(n) if k != j]
                  for i in range(1, n)]
        det += ((-1)**j) * M[0][j] * determinant(mineur)
    return det

def inverse_2x2(A):
    d = determinant(A)
    if abs(d) < 1e-12:
        raise ValueError("Matrice non inversible (det = 0)")
    return [[A[1][1]/d, -A[0][1]/d],
            [-A[1][0]/d, A[0][0]/d]]

def gauss(A, b):
    """Résolution Ax=b par élimination de Gauss"""
    n = len(A)
    # Copie augmentée [A|b]
    M = [A[i][:] + [b[i]] for i in range(n)]

    for col in range(n):
        # Trouver pivot
        pivot = max(range(col, n), key=lambda r: abs(M[r][col]))
        M[col], M[pivot] = M[pivot], M[col]
        if abs(M[col][col]) < 1e-12:
            raise ValueError("Système singulier")
        # Éliminer
        for row in range(col+1, n):
            f = M[row][col] / M[col][col]
            M[row] = [M[row][k] - f*M[col][k] for k in range(n+1)]

    # Substitution arrière
    x = [0.0] * n
    for i in range(n-1, -1, -1):
        x[i] = (M[i][n] - sum(M[i][j]*x[j] for j in range(i+1, n))) / M[i][i]
    return x

# — Tests —
A = [[2, 1, -1],
     [-3, -1, 2],
     [-2, 1, 2]]
b = [8, -11, -3]

x = gauss(A, b)
print("=== Résolution Ax = b ===")
print(f"Solution x = {[round(v, 4) for v in x]}")
# Vérification
for i in range(len(A)):
    resultat = sum(A[i][j]*x[j] for j in range(len(x)))
    print(f" Ligne {i+1}: {resultat:.4f} = {b[i]} ? {abs(resultat-b[i])<1e-9}")

A2 = [[3, 1], [2, 4]]
print(f"\ndet(A2) = {determinant(A2)}") # 3*4 - 1*2 = 10
A2_inv = inverse_2x2(A2)
afficher_matrice(A2_inv, "A2-1")

```

[PIÈGE]

Éviter d'inverser une matrice pour résoudre $Ax=b$.
 $x = A^{-1}b$ est mathématiquement correct mais numériquement instable et lent.
 Préférer toujours l'élimination de Gauss (ou les solveurs directement).

[THÉORIE]

Quand on applique une matrice A à un vecteur v , le résultat est en général un autre vecteur pointant dans une direction différente. Mais pour certains vecteurs spéciaux (vecteurs propres), A ne fait que l'étirer ou le réduire. Ces directions invariantes sont fondamentales en PCA, PageRank, etc.

[DÉFINITION]

VALEUR PROPRE λ et VECTEUR PROPRE v (non nul) :
 $Av = \lambda v$ (A transforme v en λ fois lui-même)

ÉQUATION CARACTÉRISTIQUE :

$$\det(A - \lambda I) = 0$$

Polynôme de degré n en $\lambda \rightarrow n$ valeurs propres (complexes éventuellement)

DIAGONALISATION : si A possède n vecteurs propres linéairement indépendants :

$$A = P \cdot D \cdot P^{-1}$$

D = matrice diagonale des valeurs propres

P = matrice des vecteurs propres en colonnes

[DÉMONSTRATION] $\det(A - \lambda I) = 0$ est nécessaire

$$Av = \lambda v \quad \square \quad Av - \lambda v = 0 \quad \square \quad (A - \lambda I)v = 0$$

Pour que cette équation ait une solution non nulle v ,
la matrice $(A - \lambda I)$ ne doit pas être inversible.

Donc $\det(A - \lambda I) = 0$. $\square$

[EXEMPLE]

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$

$$\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - 1 \cdot 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2) = 0$$

$$\lambda_1 = 5, \lambda_2 = 2$$

$$\text{Pour } \lambda_1 = 5 : (A - 5I)v = 0 \rightarrow \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} v = 0 \rightarrow v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\text{Pour } \lambda_2 = 2 : (A - 2I)v = 0 \rightarrow \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} v = 0 \rightarrow v_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

[CODE PYTHON]

```
import math

def valeurs_propres_2x2(A):
    """
    Valeurs propres d'une matrice 2x2 via équation caractéristique.
    det(A - λI) = λ² - tr(A)λ + det(A) = 0
    """
    trace = A[0][0] + A[1][1]
    det_A = A[0][0]*A[1][1] - A[0][1]*A[1][0]
    # Discriminant
    delta = trace**2 - 4*det_A
    if delta < 0:
        print("Valeurs propres complexes (delta < 0)")
        return None
    lam1 = (trace + math.sqrt(delta)) / 2
    lam2 = (trace - math.sqrt(delta)) / 2
    return lam1, lam2

def vecteur_propre_2x2(A, lam):
    """Vecteur propre (normalisé) pour la valeur propre lam"""
    # (A - λI)v = 0 → utiliser la première ligne
    a = A[0][0] - lam
    b = A[0][1]
    # Si a et b sont tous les deux nuls, utiliser la 2ème ligne
    if abs(a) < 1e-10 and abs(b) < 1e-10:
        a = A[1][0]
        b = A[1][1] - lam
    # Solution : v = [-b, a] (vecteur nul de la ligne)
    if abs(a) < 1e-10 and abs(b) < 1e-10:
        return [1, 0] # cas dégénéré
    v = [-b, a]
    norme = math.sqrt(v[0]**2 + v[1]**2)
    return [v[0]/norme, v[1]/norme]
```

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$

$$\text{lam1, lam2} = \text{valeurs_propres_2x2}(A)$$

$$v1 = \text{vecteur_propre_2x2}(A, \text{lam1})$$

$$v2 = \text{vecteur_propre_2x2}(A, \text{lam2})$$

```

print("=== Décomposition propre ===")
print(f"λ1 = {lam1:.4f}, v1 = [{v1[0]:.4f}, {v1[1]:.4f}]")
print(f"λ2 = {lam2:.4f}, v2 = [{v2[0]:.4f}, {v2[1]:.4f}]")

# Vérification Av = λv
def matmul_vect(A, v):
    return [sum(A[i][j]*v[j] for j in range(len(v))) for i in range(len(A))]

Av1 = matmul_vect(A, v1)
print(f"\nVérification Av1 = [{Av1[0]:.4f}, {Av1[1]:.4f}]")
print(f"λ1·v1 = [{lam1*v1[0]:.4f}, {lam1*v1[1]:.4f}]")
print(f"Égale ? {all(abs(Av1[i] - lam1*v1[i]) < 1e-9 for i in range(2))}")

```

[PIÈGE]

Les vecteurs propres sont définis à un scalaire près : si v est vecteur propre, $k \cdot v$ l'est aussi pour tout $k \neq 0$. La DIRECTION est unique, pas la norme. Toujours normaliser ou choisir une convention claire.

PARTIE 7 – ÉQUATIONS DIFFÉRENTIELLES

15. ÉQUATIONS DIFFÉRENTIELLES DU 1ER ORDRE

[THÉORIE]

Une équation différentielle relie une fonction inconnue $y(t)$ à ses dérivées. Au lieu de trouver un nombre x tel que $f(x) = 0$, on cherche une FONCTION y telle que $f(y, y', t) = 0$. Elles modélisent tout ce qui évolue dans le temps : croissance, refroidissement, circuits électriques, épidémies.

[DÉFINITION]

ÉQUATION DIFFÉRENTIELLE ORDINAIRE (EDO) du 1er ordre :
 $y'(t) = f(t, y(t))$ avec condition initiale $y(t_0) = y_0$

FORME SÉPARABLE :

$$y' = g(t) \cdot h(y) \rightarrow \int dy/h(y) = \int g(t)dt$$

FORME LINÉAIRE DU 1ER ORDRE :

$$y' + p(t) \cdot y = q(t)$$

Solution : $y(t) = e^{(-\int p)} \cdot \left[\int q \cdot e^{(\int p)} dt + C \right]$

TYPES COURANTS :

Croissance exponentielle : $y' = ky \rightarrow y(t) = y_0 \cdot e^{(kt)}$
 Refroidissement (Newton) : $y' = -k(y - T_{\text{ext}}) \rightarrow y(t) = T_{\text{ext}} + (y_0 - T_{\text{ext}}) \cdot e^{(-kt)}$
 Logistique : $y' = ry(1 - y/K)$

[DÉMONSTRATION] $y' = ky \rightarrow y(t) = y_0 \cdot e^{(kt)}$

Séparation des variables :

$$dy/y = k \cdot dt$$

Intégration :

$$\ln|y| = kt + C_1$$

$$y = C \cdot e^{(kt)} \quad (C = e^{(C_1)} \text{ constante d'intégration})$$

Condition initiale $y(0) = y_0$:

$$y_0 = C \cdot e^0 = C$$

Donc $y(t) = y_0 \cdot e^{(kt)}$. □

[EXEMPLE]

Population bactérienne, t en heures, y en milliers :

$$y' = 0.3 \cdot y, \quad y(0) = 1$$

$$\text{Solution : } y(t) = e^{(0.3t)}$$

$$\text{À } t=5\text{h : } y(5) = e^{1.5} \approx 4.48 \text{ milliers}$$

Refroidissement d'une tasse de café :

$$y' = -0.1(y - 20), \quad y(0) = 90^\circ\text{C}, \quad T_{\text{ext}} = 20^\circ\text{C}$$

$$\text{Solution : } y(t) = 20 + 70 \cdot e^{(-0.1t)}$$

$$\text{À } t=10 : y(10) = 20 + 70 \cdot e^{(-1)} \approx 45.7^\circ\text{C}$$

[CODE PYTHON]

```
import math
```

— Méthode d'Euler (résolution numérique) —————

```
def euler(f, y0, t0, tf, h=0.01):
    """
    Résolution d'EDO  $y' = f(t, y)$  par méthode d'Euler.
    Retourne listes (t_vals, y_vals).
    """
    t_vals = [t0]
    y_vals = [y0]
    t, y = t0, y0
    while t < tf:
        y = y + h * f(t, y)
        t = t + h
        t_vals.append(round(t, 10))
        y_vals.append(y)
    return t_vals, y_vals
```

— Runge-Kutta d'ordre 4 (plus précis) —————

```
def rk4(f, y0, t0, tf, h=0.1):
    """Méthode RK4 – bien plus précise qu'Euler pour même h"""
    t_vals = [t0]
    y_vals = [y0]
    t, y = t0, y0
    while t < tf - h/2:
        k1 = f(t, y)
        k2 = f(t + h/2, y + h*k1/2)
        k3 = f(t + h/2, y + h*k2/2)
        k4 = f(t + h, y + h*k3)
        y = y + h * (k1 + 2*k2 + 2*k3 + k4) / 6
        t = t + h
        t_vals.append(round(t, 10))
        y_vals.append(y)
    return t_vals, y_vals
```

— Exemple 1 : Croissance exponentielle $y' = 0.3y$ —————

```
f_exp = lambda t, y: 0.3 * y
t_euler, y_euler = euler(f_exp, y0=1, t0=0, tf=5, h=0.1)
t_rk4, y_rk4 = rk4(f_exp, y0=1, t0=0, tf=5, h=0.5)

print("=== Croissance exponentielle  $y'=0.3y$ ,  $y(0)=1$  ===")
print(f"{'t':>4} | {'Euler':>10} | {'RK4':>10} | {'Exact':>10} | {'Err RK4':>10}")
print("-" * 55)
for i, t in enumerate(range(0, 6)):
    exact = math.exp(0.3 * t)
    euler_v = y_euler[min(i*10, len(y_euler)-1)]
    rk4_v = y_rk4[min(i*2, len(y_rk4)-1)]
    err_rk4 = abs(rk4_v - exact)
    print(f"{'t':>4} | {'euler_v':>10.4f} | {'rk4_v':>10.4f} | {'exact':>10.4f} | {'err_rk4':>10.6f}")
```

— Exemple 2 : Refroidissement de Newton —————

```
T_ext = 20
k = 0.1
f_ref = lambda t, y: -k * (y - T_ext)
solution_exacte = lambda t: T_ext + 70 * math.exp(-k * t)

t_vals, y_vals = rk4(f_ref, y0=90, t0=0, tf=30, h=1)

print("\n=== Refroidissement (café à 90°C,  $T_{ext}=20^{\circ}\text{C}$ ) ===")
for i in range(0, min(len(t_vals), 31), 5):
    t = t_vals[i]
    print(f"t={t:5.1f}h :  $T_{num}=\{y\_vals[i]:6.2f\}^{\circ}\text{C}$ ,  $T_{exact}=\{solution\_exacte(t):6.2f\}^{\circ}\text{C}$ ")
```

— Exemple 3 : Croissance logistique $y' = ry(1-y/K)$ —————

```
r, K = 0.5, 100
f_log = lambda t, y: r * y * (1 - y / K)
t_log, y_log = rk4(f_log, y0=5, t0=0, tf=20, h=0.2)

print("\n=== Croissance logistique ( $r=0.5$ ,  $K=100$ ,  $y_0=5$ ) ===")
for i in range(0, len(t_log), 10):
    print(f"t={t_log[i]:5.1f} : population = {y_log[i]:6.2f}")
```

[PIÈGE]

La méthode d'Euler est simple mais accumule des erreurs rapidement.
Avec un pas h grand, la solution diverge. Préférer RK4 qui est d'ordre 4 :
l'erreur est proportionnelle à h^4 (au lieu de h pour Euler).
Réduire h de moitié réduit l'erreur Euler de moitié, mais RK4 de 16 fois.

16. ÉQUATIONS DIFFÉRENTIELLES DU 2ème ORDRE

[THÉORIE]

Les EDO du 2ème ordre apparaissent partout en physique : oscillation d'un ressort, circuit LC, onde, pendule. Elles font intervenir y , y' et y'' .
La solution générale est une combinaison linéaire de deux solutions fondamentales (espace vectoriel de dimension 2).

[DÉFINITION]

FORME GÉNÉRALE :

$$y'' + p(t) \cdot y' + q(t) \cdot y = r(t)$$

CAS HOMOGENE LINEAIRE À COEFFICIENTS CONSTANTS ($r=0$) :

$$ay'' + by' + cy = 0$$

On cherche $y = e^{\lambda t} \rightarrow a\lambda^2 + b\lambda + c = 0$ (équation caractéristique)

DISCRIMINANT $\Delta = b^2 - 4ac$:

$\Delta > 0$: deux racines réelles $\lambda_1 \neq \lambda_2 \rightarrow y = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t}$

$\Delta = 0$: racine double $\lambda \rightarrow y = (C_1 + C_2 t) e^{\lambda t}$

$\Delta < 0$: racines complexes $\alpha \pm i\omega \rightarrow y = e^{\alpha t} (C_1 \cos(\omega t) + C_2 \sin(\omega t))$

OSCILLATEUR HARMONIQUE :

$$y'' + \omega^2 y = 0 \quad (\text{ex: ressort sans frottement})$$

Solution : $y(t) = A \cdot \cos(\omega t) + B \cdot \sin(\omega t)$

OSCILLATEUR AMORTI :

$$y'' + 2\gamma y' + \omega_0^2 y = 0$$

$\gamma < \omega_0$: sous-amorti (oscillations qui s'amortissent)

$\gamma = \omega_0$: critique

$\gamma > \omega_0$: sur-amorti (retour exponentiel sans oscillation)

[DÉMONSTRATION] $y'' + \omega^2 y = 0 \rightarrow y = A \cdot \cos(\omega t) + B \cdot \sin(\omega t)$

On cherche $y = e^{\lambda t}$:

$$\lambda^2 + \omega^2 = 0 \rightarrow \lambda = \pm i\omega$$

Racines complexes conjuguées : $\alpha = 0, \beta = \omega$

Solution réelle : $y = e^{(0 \cdot t)} (C_1 \cos(\omega t) + C_2 \sin(\omega t)) = C_1 \cos(\omega t) + C_2 \sin(\omega t)$. $\square$

[CODE PYTHON]

```
import math

# Réduction d'ordre : y'' = f(t,y,y') → système du 1er ordre
# Posons u = [y, y'] = [u0, u1]
# Alors u' = [u1, f(t, u0, u1)]

def rk4_système(F, u0, t0, tf, h=0.05):
    """
    RK4 pour système d'EDO du 1er ordre : u' = F(t, u)
    u0 = liste des conditions initiales
    F(t, u) = liste des dérivées
    """
    t_vals = [t0]
    u_vals = [u0[:]]
    t, u = t0, u0[:]
    while t < tf - h/2:
        k1 = F(t, u)
        k2 = F(t + h/2, [u[i] + h*k1[i]/2 for i in range(len(u))])
        k3 = F(t + h/2, [u[i] + h*k2[i]/2 for i in range(len(u))])
        k4 = F(t + h, [u[i] + h*k3[i] for i in range(len(u))])
        u = [u[i] + h*(k1[i] + 2*k2[i] + 2*k3[i] + k4[i])/6
              for i in range(len(u))]
        t = t + h
        t_vals.append(t)
        u_vals.append(u[:])
    return t_vals, u_vals

# — Oscillateur harmonique y'' + ω²y = 0 —
```

```

omega = 2 * math.pi # période = 1
F_harm = lambda t, u: [u[1], -omega**2 * u[0]] # [y', y'']

t_vals, u_vals = rk4_systeme(F_harm, u0=[1, 0], t0=0, tf=2, h=0.01)

print("=== Oscillateur harmonique y''+4π²y=0, y(0)=1, y'(0)=0 ===")
print(f"{'t':>5} | {'y_num':>10} | {'y_exact':>10} | {'erreur':>10}")
print("-" * 44)
for i in range(0, len(t_vals), len(t_vals)//10):
    t = t_vals[i]
    y_n = u_vals[i][0]
    y_ex = math.cos(omega * t) # solution exacte
    print(f"{'t':>5.3f} | {'y_n':>10.6f} | {'y_ex':>10.6f} | {'abs(y_n-y_ex):>10.2e}")

# — Oscillateur amorti y'' + 2γy' + ω₀²y = 0 —————
omega0 = 2 * math.pi
gamma = 0.5

F_amorti = lambda t, u: [
    u[1],
    -2*gamma*u[1] - omega0**2 * u[0]
]

t_amorti, u_amorti = rk4_systeme(F_amorti, u0=[1, 0], t0=0, tf=3, h=0.01)

print("\n=== Oscillateur amorti (γ=0.5, ω₀=2π) ===")
print("Position au cours du temps :")
for i in range(0, len(t_amorti), len(t_amorti)//8):
    t = t_amorti[i]
    y = u_amorti[i][0]
    amp = abs(y)
    bar = "█" * int(amp * 20)
    print(f"t={t:5.2f} : y={y:+.4f} {bar}")

```

[PIÈGE]

Un problème du 2ème ordre a besoin de DEUX conditions initiales : $y(t_0)$ et $y'(t_0)$.
 Sans les deux, on ne peut pas déterminer la solution unique.
 En physique : position initiale ET vitesse initiale sont toutes deux nécessaires.

RÉSUMÉ – LES RÈGLES D'OR

STAT. DESCRIPTIVES

Moyenne = centre de gravité. Médiane = robuste aux outliers.
 Variance mesure l'étalement. Toujours utiliser n-1 en inférence.

PROBABILITÉS

Bayes = mettre à jour nos croyances avec de nouvelles données.
 $P(A|B) \neq P(B|A)$ – erreur logique classique.

INFÉRENCE STATISTIQUE

IC = plage plausible pour le paramètre inconnu.
 p-valeur $\neq$ probabilité que H_0 est vraie. Attention à l'abus.

CORRÉLATION & RÉGRESSION

Corrélation $\neq$ causalité.
 R^2 mesure la qualité du modèle. Ne pas extrapoler.

SÉRIES TEMPORELLES

Décomposer avant de prévoir : tendance + saisonnalité + bruit.
 La moyenne mobile et le lissage exponentiel estiment la tendance.

MATRICES

$AB \neq BA$. Dimensions compatibles : $(m \times p) \cdot (p \times n) = (m \times n)$.
 $\det=0 \rightarrow$ non inversible. Gauss est plus stable que l'inverse.
 Valeurs propres = directions invariantes par la transformation.

ÉQUATIONS DIFFÉRENTIELLES

$y'=ky \rightarrow$ croissance/décroissance exponentielle (modèle universel).
 Euler est simple mais imprécis. RK4 : erreur en h^4 .
 Ordre 2 $\rightarrow$ 2 conditions initiales obligatoires.

```
#####  
#  
# CE QUE TU MAÎTRISES MAINTENANT :  
#  
# ✓ Statistiques descriptives : moyenne, médiane, mode, variance, IQR  
# ✓ Probabilités : axiomes, conditionnelle, Bayes, distributions  
# ✓ Statistiques inférentielles : IC, tests d'hypothèses, p-valeur, erreurs I/II  
# ✓ Corrélation & Régression : Pearson, Spearman, moindres carrés,  $R^2$   
# ✓ Séries temporelles : décomposition, moyenne mobile, lissage expo  
# ✓ Matrices : opérations, déterminant, inverse, Gauss  
# ✓ Valeurs propres : équation caractéristique, diagonalisation  
# ✓ Équations différentielles : séparables, linéaires, Euler, RK4, ordre 2  
#  
# PROCHAINE ÉTAPE NATURELLE :  
# → Analyse (séries de Taylor, intégrales multiples)  
# → Optimisation (gradient, convexité, Lagrange)  
# → Algèbre linéaire avancée (SVD, décomposition QR)  
# → Probabilités avancées (chaînes de Markov, processus stochastiques)  
#  
#####
```